

Algorithm Analysis: Overview and Objectives

Counting steps, growth rates, and Big-O

1. What this unit is about

So far this semester, an algorithm has been judged mostly by whether it works. This unit asks a different question: given two working algorithms that solve the same problem, which one is actually better, and how would we know?

A tempting answer is to run both and time them. This unit starts by showing why that answer is weak. A stopwatch measures one run, on one machine, with one input, on one day. It does not tell you what happens when the input triples in size, and it does not transfer from one computer to another. What we want instead is a way to describe how the amount of work an algorithm does grows as the input grows, independent of hardware.

That is what algorithm analysis gives us. We count the operations an algorithm performs as a function of the input size, usually called n , rather than the seconds it takes. Once we have that count, we care about only its dominant term, since that term is what decides the algorithm's behavior once n gets large. **Big-O notation** is the standard shorthand for naming that dominant term, and it is the vocabulary you will use to talk about every sorting algorithm, tree operation, and graph algorithm for the rest of the course.

If a couple of these ideas already sound familiar, that is on purpose. Earlier this semester, a couple of assignments used a short, deliberately incomplete preview: an $O(1)$ operation costs the same no matter the input size, an $O(n)$ operation's cost grows in direct proportion to it. This unit is where that preview gets its full treatment, along with the rest of the standard growth rates and the rules for working with them.

This is a single, dense 50-minute session, since the plan is to move into sorting algorithms right after. Class time is a slide-driven walkthrough of two worked examples, counted by hand, rather than a mix of board work and slides. The notes document that accompanies this handout carries the same two examples with the full derivation shown, so you have a written copy of every step to study from afterward.

2. Learning objectives

By the end of this unit you should be able to:

Remember and understand

- Explain why comparing algorithms by measured run time (for example, in nanoseconds) is unreliable, and state what algorithm analysis measures instead.
- Recall that we count a characteristic operation as a function of input size n , rather than counting

literal seconds.

- Define best-case, worst-case, and average-case running time.
- State what Big-O notation describes: an upper bound on how an algorithm's step count grows as n grows, expressed by its dominant term.
- Recall the common growth-rate classes and their names: constant, logarithmic, linear, loglinear (linearithmic), quadratic, cubic, and exponential.

Apply

- Given a short piece of pseudocode, count how many times its characteristic operation executes as a function of n .
- Classify a single loop as $O(n)$ and a pair of nested loops over the same input as $O(n^2)$, by recognizing the pattern rather than recounting from scratch each time.
- Apply the two simplification rules, drop lower-order terms and drop constant multiplicative factors, to reduce a step-count formula to its Big-O class.
- Combine the growth rates of two parts of an algorithm run one after another, using $O(f(n)) + O(g(n)) = O(f(n) + g(n))$.

Analyze

- Compare two algorithms that solve the same problem by their growth rate rather than by a single measured run, including cases where the "slower-looking" algorithm on small input is the better choice on large input.
- Explain why counting comparisons alone can be misleading when the algorithms being compared do different amounts of work per comparison.
- Identify the best, worst, and average case of sequential search, and explain why the worst and average cases are usually the ones we report.
- Explain, in plain language, why an algorithm that repeatedly cuts its input in half (such as binary search) grows logarithmically rather than linearly.

Evaluate

- Judge which of two algorithms is the better general-purpose choice based on its Big-O class, and state the conditions under which the other one might still be preferable (for example, a small or already-favorable input).
- Critique a benchmark-only argument for "algorithm X is faster than algorithm Y," given what this unit covers about the limits of timing a single run.